ARCHITECTURE & DESIGN DOCUMENT

Annex

Extra desktop, no extra hardware.

A Rust system that turns a Windows PC into a wireless **extended** monitor for a Mac. No HDMI, no cables.

Rust · macOS host · Windows / browser client · WebRTC · ScreenCaptureKit · VideoToolbox · CGVirtualDisplay · LAN / Wi-Fi

---

Project name: Annex · **Version:** 1.0 (design) · **Date:** 24 July 2026

Goal: Mac is the primary computer; a second laptop/PC becomes a real, extended second desktop over the local network.

Audience: the developer(s) building it. Assumes general programming ability; explains all macOS/Rust specifics.

Table of Contents

1. Executive Summary

1.1 What we are building

Annex is a two-part application. On the Mac (the *host*) a Rust program creates a **virtual monitor** that macOS treats as a genuine second display, and you can drag windows onto it exactly as if a physical monitor were plugged in. The Rust program captures that virtual monitor's pixels, encodes them to video, and streams them over Wi-Fi/LAN to a *client* running on the Windows laptop. The client renders the video full-screen, so the Windows laptop **becomes** the second monitor. No HDMI cable, no capture card, just the two machines on the same network.

Version 1's client is an ordinary **web browser** (zero install on Windows); a native Rust client is a later milestone. Optionally, the client can send mouse/keyboard input back to the Mac so the second screen is interactive, not just a viewer.

Why this is fundamentally possible: every piece already exists in shipping software: **DeskPad** and **BetterDummy** prove the virtual-display trick; **Deskreen** proves the browser-streaming trick. Annex combines both, in one Rust codebase, as a true *extended* (not mirrored) display.

1.2 The core insight: this is two separate problems

The single most important thing to understand before writing any code is that the project is **two very different sub-systems** bolted together. They share almost no techniques and differ enormously in difficulty.

Half	Job	Mechanism	Difficulty
Half 1: Virtual display	Make macOS believe a monitor is attached, so windows can live there	Private, undocumented `CGVirtualDisplay` APIs inside CoreGraphics	**Hard** (private API)
Half 2: Capture + stream	Grab that display's pixels, encode, send to Windows, render	ScreenCaptureKit → VideoToolbox → WebRTC → browser	**Well-trodden** (standard video streaming)

If we build only Half 2, we have "our own Deskreen" that mirrors the main screen and relies on a third-party tool (BetterDisplay) for the extend. If we build **both**, Annex is a complete, self-contained product. The roadmap (§14) deliberately sequences Half 2 first (it is de-riskable and demoable early) and folds in Half 1 once the streaming pipeline works end-to-end.

2. Goals, Non-Goals & Success Criteria

2.1 Goals (v1)

- **Mac as host:** the Mac is the primary computer whose desktop is extended.
- **True extend, not mirror:** the Windows screen is a *new* desktop area, not a copy of the Mac's screen.
- **Wireless over LAN:** both machines on the same Wi-Fi/router; no HDMI, no capture hardware.
- **Zero-install client (v1):** Windows just opens a URL in a browser.
- **One language:** the host and (later) native client are both Rust.
- **Usable latency:** smooth for reference material, docs, dashboards, video playback; target < 60 ms glass-to-glass on good Wi-Fi.
- **Configurable resolution & HiDPI** for the virtual display.

2.2 Non-goals (explicitly out of scope for v1)

- **Internet / NAT traversal.** LAN only. No STUN/TURN servers, no cloud relay.
- **Mac App Store distribution.** Ruled out deliberately, not reluctantly. App Review rejects private APIs, and we would rather have the virtual display than the storefront. We ship a notarized Developer-ID app instead (§11), which is how every comparable tool distributes. Using `CGVirtualDisplay` is therefore an accepted, deliberate choice rather than a risk to be mitigated away.
- **Audio** forwarding.
- **The reverse direction** (Windows host → Mac client).
- **Interactive input** is a *phase-2* goal, not v1.
- **Multi-client / multi-monitor fan-out**, designed-for but not built in v1.

2.3 Success criteria (definition of done for v1)

1. Launch host on Mac → a new display appears in **System Settings → Displays**.
2. Open the host's URL in a Windows browser on the same network → the virtual display's contents appear full-screen.
3. Drag a window from the Mac's main screen onto the virtual display → it shows up on the Windows laptop.
4. Motion (scrolling, a playing video) is smooth (≥ 30 fps) with sub-100 ms perceived lag on Wi-Fi.
5. Closing the host cleanly removes the virtual display (no ghost monitors left behind).

3. Technical Background & Key Constraints

3.1 The macOS media stack we depend on

Concern	API / Framework	Public?	Notes
Create virtual monitor	`CGVirtualDisplay*` (CoreGraphics)	❌ Private	4 undocumented Obj-C classes. No kext, no DriverKit, no root, no SIP-disable.
Capture a display's pixels	**ScreenCaptureKit** (`SCStream`)	✅ Public	macOS 12.3+. GPU-accelerated, delivers `CVPixelBuffer` . Requires Screen Recording permission.
Hardware video encode	**VideoToolbox** (`VTCompressionSession`)	✅ Public	C API. H.264 & HEVC on the media engine. Accepts `CVPixelBuffer` directly (near zero-copy).
Real-time transport	**WebRTC** (webrtc-rs)	✅ Pure Rust	No native libwebrtc build. Congestion control, packet loss recovery, browser-compatible.
Input injection (phase 2)	`CGEvent` (CoreGraphics)	✅ Public	Requires Accessibility permission. Maps client input → Mac events.

The one true difficulty: `CGVirtualDisplay` is undocumented. It is stable enough that DeskPad/BetterDummy rely on it across many macOS releases, but Apple can change its shape between versions. We isolate **all** private-API contact inside a single crate so breakage is contained to one file (§5.1, §13).

3.2 Why a virtual display is required at all

Screen-sharing tools can only capture pixels that macOS is *already rendering somewhere*. To get a *new* desktop area (extend), some display must exist for the windows to occupy. A physical monitor does this via HDMI/EDID; a "dummy plug" fakes it in hardware; `CGVirtualDisplay` fakes it in **software**. Once the fake display exists, capturing it and streaming it produces a genuine extended second monitor.

3.3 Constraints that shape the whole design

- **Run loop affinity:** ScreenCaptureKit, AppKit (tray UI), and the virtual-display callbacks want a thread with a Core Foundation run loop, i.e. the **main thread**. WebRTC/async must live on separate (tokio) threads and communicate via channels.
- **Zero-copy path:** ScreenCaptureKit outputs `CVPixelBuffer` ; VideoToolbox consumes `CVPixelBuffer` . Keeping frames on the GPU from capture → encode avoids expensive CPU copies.
- **Browser compatibility:** because v1's client is a browser, the encoder must emit a browser-decodable codec/profile. **H.264 (Constrained Baseline / Main, Annex-B)** is the safe universal choice; HEVC is an optimization for capable clients.
- **LAN-only:** WebRTC ICE only needs **host candidates**; no STUN/TURN. Simplifies signaling enormously.

4. High-Level Architecture

4.1 System context

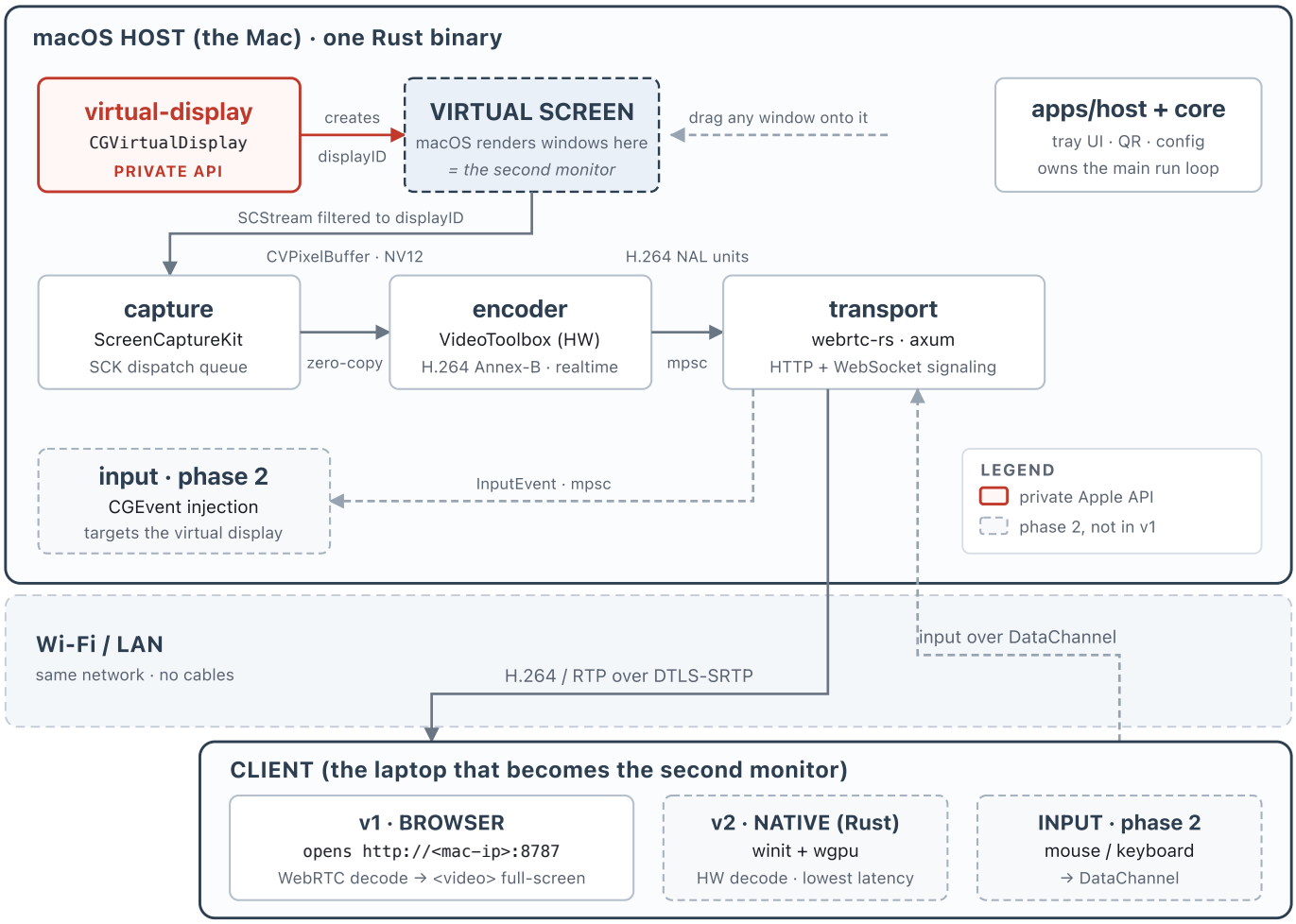


Figure 1. System context. Red border marks the single crate that touches private Apple APIs; dashed boxes are phase 2. The `displayID` produced by `virtual-display` is the handle `capture` uses to open its `SCStream`.

4.2 Process model

- **Host process (macOS):** a single native Rust binary, presented as a menu-bar/tray app. It owns the virtual display, the capture+encode pipeline, an embedded HTTP server (serves the client page), a WebSocket signaling endpoint, and one WebRTC peer connection per connected client.
- **Client (v1 = browser):** a static HTML/JS page (served by the host) that establishes a WebRTC session and paints the incoming video track. Nothing to install on Windows.
- **Client (v2 = native Rust):** a `winit` + `wgpu` window that decodes the stream and renders it, for lower latency and a cleaner full-screen experience. Same signaling protocol.

4.3 End-to-end data flow (steady state)

1. macOS composits the windows sitting on the virtual display into its framebuffer.

2. **ScreenCaptureKit** delivers each new frame as a `CVPixelBuffer` on its dispatch queue.
3. The frame is handed (zero-copy) to **VideoToolbox**, which emits an H.264 access unit (NAL units).
4. Encoded samples cross an **mpsc channel** into the tokio world.
5. **webrtc-rs** packetizes them into RTP and sends over the peer connection.
6. The browser's WebRTC stack depacketizes, decodes (hardware), and renders to a `<video>` element.
7. *(phase 2)* the client captures pointer/key events, sends them on a **DataChannel**; the host maps coordinates into the virtual display and injects them with `CGEvent`.

5. Component Design

Each component is a Rust crate with a narrow, testable surface. Below: responsibility, public API sketch, and the key implementation notes/gotchas.

5.1 `virtual-display` : the private-API firewall

Responsibility: create/destroy a macOS virtual monitor and expose its `CGDirectDisplayID`. This is the **only** crate that touches private Apple APIs. It wraps the four Obj-C classes (`CGVirtualDisplayDescriptor`, `…Mode`, `…Settings`, `CGVirtualDisplay`) via `objc2` runtime class lookup, and uses RAII so dropping the handle removes the monitor.

```
pub struct VirtualDisplayConfig {
    pub name: String,
    pub width: u32, pub height: u32, pub refresh_hz: f64,
    pub hidpi: bool,                 // true → Retina backing store
    pub size_mm: (f64, f64),         // physical size → controls default DPI
}

pub struct VirtualDisplay { /* retains the CGVirtualDisplay obj-c object */ }

impl VirtualDisplay {
    /// Looks up CGVirtualDisplay* classes at runtime, builds descriptor +
    /// settings, calls initWithDescriptor: / applySettings:.
    pub fn create(cfg: &VirtualDisplayConfig) -> Result<Self, VdError>;
    /// CGDirectDisplayID: hand this to the capture crate.
    pub fn display_id(&self) -> u32;
}
impl Drop for VirtualDisplay { /* release obj-c object → monitor disappears */ }
```

Gotchas: (1) the `CGVirtualDisplay` instance *is* the monitor, so it must be kept alive for the whole session; (2) the descriptor needs a valid **dispatch queue** (main queue works), since some macOS versions insist on it; (3) the `terminationHandler` is an Obj-C **block** (`block2` crate), can be a no-op in v1; (4) copy the exact header from DeskPad/BetterDummy, do not transcribe field signatures from memory (see App. A).

5.2 `capture` : ScreenCaptureKit source

Responsibility: given a `display_id` , open an `SCStream` filtered to that display and deliver frames as `CVPixelBuffer` s to a callback/channel. Handles resolution, pixel format (BGRA or NV12), target FPS, and start/stop.

```
pub struct CaptureConfig { pub display_id: u32, pub fps: u32, pub pixel_format: PixFmt }
pub struct Capturer { /* holds SCStream + delegate */ }

impl Capturer {
    pub fn start(cfg: CaptureConfig, sink: FrameSink) -> Result<Self, CaptureError>;
    pub fn stop(self);
}
// FrameSink receives (CVPixelBuffer, presentation_timestamp) on SCK's queue.
```

Notes: resolve the `SCDisplay` whose `displayID` equals our virtual display via `SCShareableContent` . Request **NV12** output when feeding VideoToolbox (native encoder input, avoids a color convert). Set `minimumFrameInterval` for the target FPS. The delegate callback runs on a dispatch queue, so bridge into Rust and forward without blocking.

5.3 `encoder` : VideoToolbox H.264/HEVC

Responsibility: compress `CVPixelBuffer` s into an elementary stream of NAL units suitable for WebRTC. Configured for **real-time, low-latency** operation.

```
pub struct EncoderConfig {
    pub width:u32, pub height:u32, pub fps:u32,
    pub bitrate_kbps:u32, pub codec:Codec /* H264 | HEVC */,
    pub keyframe_interval:u32,
}
pub struct Encoder { /* VTCompressionSession */ }
impl Encoder {
    pub fn new(cfg:EncoderConfig, out:EncodedSink) -> Result<Self, EncError>;
    pub fn encode(&self, frame:&CVPixelBuffer, pts:Timestamp);
    pub fn request_keyframe(&self);   // on new client / packet loss
    pub fn set_bitrate(&self, kbps:u32);  // react to WebRTC congestion signals
}
```

Critical VideoToolbox settings for low latency: `RealTime = true` , `AllowFrameReordering = false` (no B-frames), `ProfileLevel = Baseline/Main AutoLevel` , `MaxKeyFrameInterval` tuned, and **Annex-B** output so WebRTC's H.264 packetizer can consume it. Emit SPS/PPS with each keyframe (in-band) for mid-stream client joins.

5.4 `transport` : WebRTC peer + signaling

Responsibility: (a) serve the client page over HTTP; (b) run a WebSocket signaling channel to exchange SDP + ICE; (c) manage a `RTCPeerConnection` per client with one video track (and a DataChannel for phase-2

input); (d) push encoded samples into the video track.

```
// signaling messages (JSON over WebSocket)
enum Signal { Offer(Sdp), Answer(Sdp), Ice(Candidate), Hello{token:String} }

pub struct Session { pc: RTCPeerConnection, video: Arc<TrackLocalStaticSample> }
impl Session {
    pub async fn accept(ws:WebSocket, cfg:&RtcConfig) -> Result<Session, RtcError>;
    pub async fn write_sample(&self, nal:&[u8], dur:Duration); // feed encoder output
    pub fn on_input(&self, cb: impl Fn(InputEvent));           // phase-2 DataChannel
}
```

Notes: since it is LAN-only, configure ICE with **no STUN/TURN**, since host candidates connect directly. Use `TrackLocalStaticSample` with the H.264 payloader; webrtc-rs handles RTP + NACK + PLI. On a PLI (picture-loss indication) from the client, call `encoder.request_keyframe()`. Prefer an async HTTP framework (`axum`) for both the static page and the WebSocket upgrade on one port.

5.5 `web/client`: the browser client (v1)

Responsibility: a single static page. Opens a WebSocket to the host, performs the WebRTC handshake, attaches the remote track to a full-screen `<video autoplay playsinline>`, and (phase 2) forwards pointer/keyboard events. ~150 lines of vanilla JS; no framework needed.

```
const pc = new RTCPeerConnection({ iceServers: [] });   // LAN → no STUN/TURN
pc.ontrack = e => { videoEl.srcObject = e.streams[0]; };
const ws = new WebSocket(`ws://${location.host}/signal`);
// exchange offer/answer + ICE over ws … then video.requestFullscreen()
```

5.6 `input`: CGEvent injection (phase 2)

Responsibility: translate `InputEvent`s from the client into macOS events targeted at the virtual display's coordinate space, via `CGEventCreateMouseEvent` / `…KeyboardEvent` + `CGEventPost`. Requires the user to grant **Accessibility** permission. Coordinate mapping must account for the virtual display's origin in the global display arrangement and HiDPI scale factor.

5.7 `core` + `apps/host`: orchestration & UI

core holds shared types (config, errors, the frame/sample structs, protocol enums) and the pipeline wiring. **apps/host** is the actual binary: it owns the main-thread run loop, builds a tray menu (`tray-icon` / `muda`), shows the connect URL + a QR code, starts the tokio runtime, and glues virtual-display → capture → encoder → transport together.

6. Workspace & Crate Layout

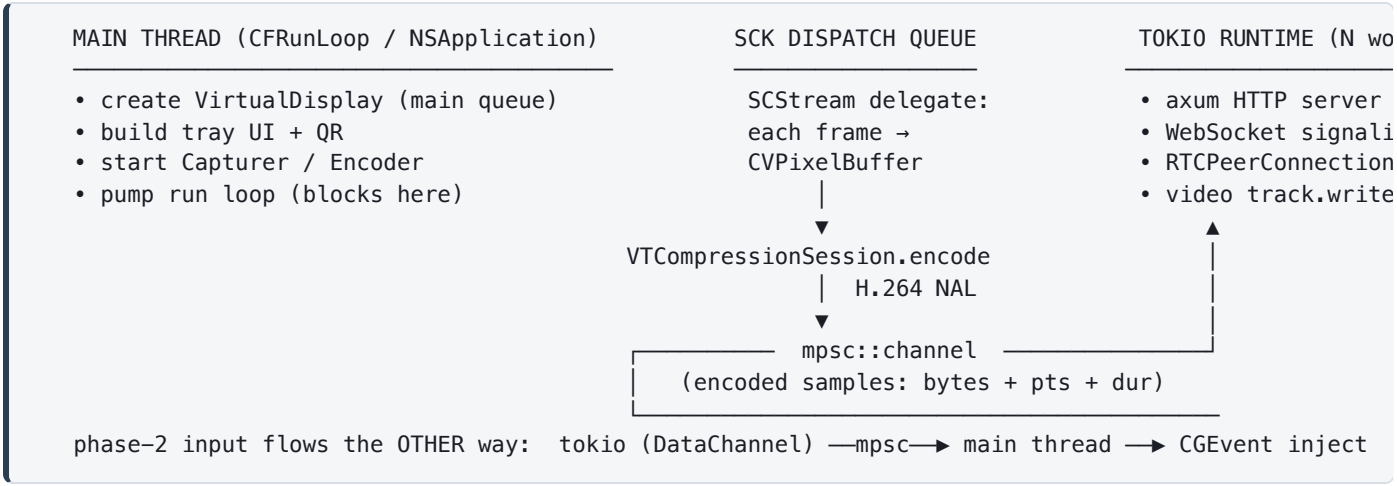
```
annex/
├─ Cargo.toml                  # [workspace] members
├─ rust-toolchain.toml         # pin stable toolchain
├─ crates/
│  ├─ core/                    # shared types, config, errors, pipeline glue
│  ├─ virtual-display/         # ← ONLY crate using private CGVirtualDisplay (objc2)
│  ├─ capture/                 # ScreenCaptureKit  → CVPixelBuffer frames
│  ├─ encoder/                 # VideoToolbox      → H.264 / HEVC NAL units
│  ├─ transport/              # webrtc-rs peer + axum HTTP/WebSocket signaling
│  └─ input/                   # (phase 2) CGEvent injection
├─ apps/
│  ├─ host/                    # macOS binary: tray UI, run loop, wires it all
│  └─ client-native/           # (phase 2) winit + wgpu + webrtc-rs client
└─ web/
   └─ client/                  # index.html + client.js  (v1 browser client, served by host)
```

Why a workspace of small crates? The private-API blast radius is confined to `virtual-display`; each crate is independently unit-testable; the media crates (macOS-only) are cleanly separated from the cross-platform `transport` / `core`, which the future Windows native client reuses.

Crate	Platform	Touches private API?	Reused by native client?
core	all	no	✅
virtual-display	macOS	**yes**	no (host only)
capture	macOS	no	no (host only)
encoder	macOS	no	no (host only)
transport	all	no	✅
input	macOS	no	no

7. Concurrency & Threading Model

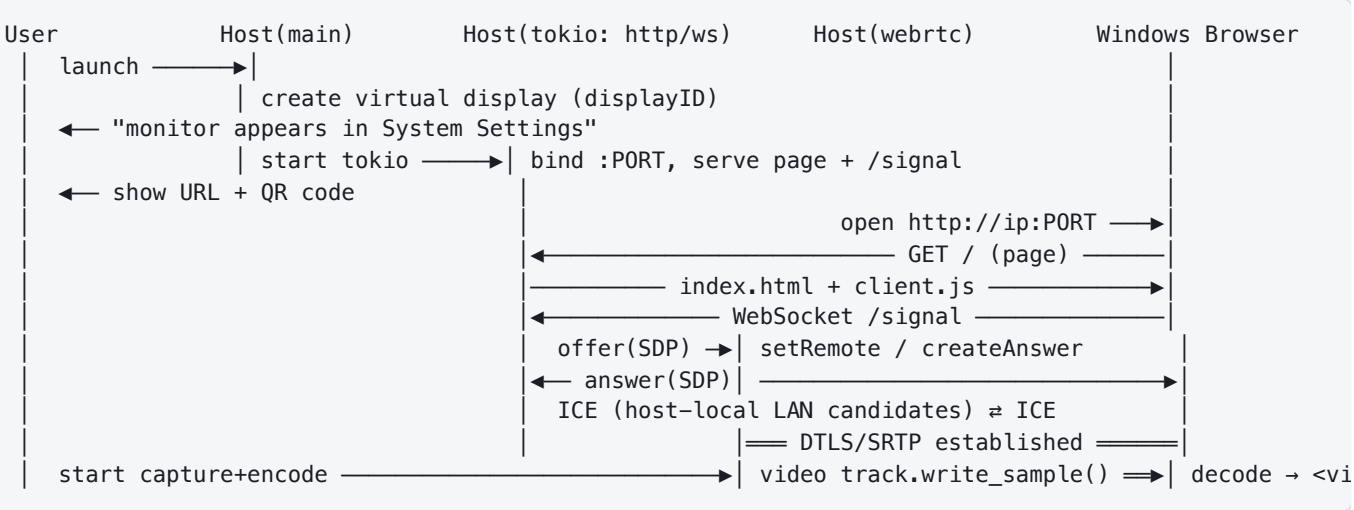
The trickiest systems detail. macOS frameworks impose **thread affinity**; Rust async imposes its own. The design keeps a strict separation between the **main run-loop world** (Apple frameworks) and the **tokio world** (networking), joined by lock-free channels.



- **Main thread** owns everything AppKit/CoreGraphics/ScreenCaptureKit and runs the run loop.
- **ScreenCaptureKit** calls the frame delegate on its own dispatch queue; we encode there (or hop to a dedicated encode thread) and push bytes onto an `mpsc`.
- **Tokio** runs on background threads (spawned before the run loop takes over the main thread); it drains the channel and writes to the WebRTC track. Networking never blocks capture.
- **Back-pressure:** a bounded channel; if the network stalls, drop the oldest frame and ask the encoder for a keyframe rather than queueing latency.

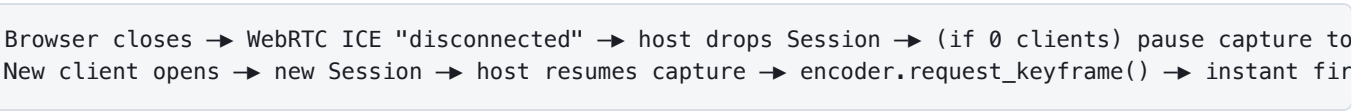
8. Sequence Diagrams

8.1 Startup & connection handshake



8.2 Steady-state frame pipeline (per frame, ~16 ms @ 60 fps)

8.3 Client disconnect / reconnect



9. Key Data Structures & APIs

The shared vocabulary lives in `core`. Everything else depends on these.

```
// ---- core::config ----
pub struct HostConfig {
    pub display: VirtualDisplayConfig,   // §5.1
    pub encoder: EncoderConfig,          // §5.3
    pub bind_addr: SocketAddr,           // e.g. 0.0.0.0:8787
    pub auth_token: Option<String>,      // simple shared secret (§11)
    pub allow_input: bool,               // phase-2 gate
}

// ---- core::frame ----
pub struct EncodedSample { pub data: Bytes, pub pts: i64, pub dur_us: u32, pub keyframe: bool }

// ---- core::protocol (client ⇄ host over WebSocket, JSON) ----
pub enum ClientMsg { Hello{ token: Option<String> }, Answer(String), Ice(IceCandidate),
                     Input(InputEvent) /* phase 2, or via DataChannel */ }
pub enum HostMsg   { Offer(String), Ice(IceCandidate), Config{ w:u32, h:u32, fps:u32 }, Error(String) }

// ---- core::input (phase 2) ----
pub enum InputEvent {
    MouseMove{ x:f64, y:f64 },  MouseButton{ btn:u8, down:bool },
    Scroll{ dx:f64, dy:f64 },   Key{ code:u32, down:bool, mods:u32 },
}

// ---- pipeline handle ----
pub struct Pipeline { /* owns VirtualDisplay + Capturer + Encoder + transport tx */ }
impl Pipeline {
    pub fn start(cfg: HostConfig) -> Result<Pipeline, Error>;
    pub fn connect_url(&self) -> String;      // for the tray UI / QR
    pub fn shutdown(self);                     // tears down cleanly (removes VD)
}
```

10. Performance & Latency Budget

At 60 fps each frame owns a **16.6 ms** slot; at 30 fps, 33 ms. "Glass-to-glass" latency (a change on the Mac appearing on the Windows screen) is the sum of pipeline stages plus buffering. Realistic targets on a decent 5 GHz Wi-Fi link:

Stage	Typical	Levers to reduce it
Capture (ScreenCaptureKit)	1–3 ms	NV12 output, cap FPS, dirty-region hints
Encode (VideoToolbox HW)	3–8 ms	RealTime, no B-frames, no reordering, sane bitrate
Packetize + queue	<1 ms + queue	bounded channel, drop-oldest back-pressure
Network (Wi-Fi LAN)	2–10 ms	5 GHz / Wi-Fi 6, low channel congestion, wired > wireless
Jitter buffer + decode (browser)	5–16 ms	low-latency playback, small jitter buffer
Total (glass-to-glass)	**~30–60 ms**	Ethernet/USB-tether pushes toward the low end

Measured 27 July 2026, macOS 26.5: the virtual display is 1x, not Retina. `CGDisplayModeGetPixelWidth` equals `CGDisplayModeGetWidth` for the virtual display, so there is **no HiDPI backing store**. That holds with `hiDPI` set to 0 or 1, and with a mode of either 1920x1080 or 3840x2160: all four combinations report **1920x1080 points and 1920x1080 backing pixels**. Two consequences. **Capture at scale 1.** Requesting a 2x buffer costs four times the pixels and bandwidth and returns a bicubic upscale, differing from an upscaled 1x frame by a mean of 0.85/255, which is resampling noise. And separately, the requested **mode dimensions currently have no effect**: macOS lands on 1920x1080 whatever we ask for. BetterDummy offers a whole ladder of modes rather than the single one we apply, which is the likely fix and is M6 work (resolution picker). Nothing before M6 depends on it.

Measured 27 July 2026: the frame duration handed to WebRTC must be the real one. The receiver advances its RTP clock by `duration × 90000` per sample, and paces playback from that clock. ScreenCaptureKit emits **only on change**, so real intervals vary enormously; reporting a nominal 1/fps advances the receiver's clock far more slowly than wall time, and its jitter buffer grows without bound to compensate. Measured effect of getting this wrong: **249 ms per frame of jitter buffer, 8.5 seconds of freezes in a 14 second window, and 8 fps delivered out of 30**. With the duration computed from consecutive capture timestamps: **15 ms, zero freezes, 29 fps**. Note the build type was almost irrelevant here, an unoptimised build with the fix measured 19 ms, so this is a correctness bug and not a performance one.

Bandwidth: 1080p60 at H.264 real-time is roughly **8–20 Mbit/s** depending on motion. Static desktops compress to almost nothing; video/scrolling spikes it. WebRTC's congestion control adapts bitrate automatically, so wire that signal to `encoder.set_bitrate()`.

- **Idle efficiency:** ScreenCaptureKit only emits frames on change, so a still desktop costs ~0.
- **Motion smoothness** matters more than resolution for perceived quality; prefer 60 fps at a tuned bitrate over 4K at low fps.
- **HEVC** cuts bitrate ~30–40% but only for clients that can decode it, so keep H.264 as the fallback.

11. Security, Permissions & Privacy

- **Screen Recording permission** (TCC): required by ScreenCaptureKit. macOS prompts on first capture; the app appears under **System Settings → Privacy & Security → Screen Recording**.
- **Accessibility permission**: required only for phase-2 input injection (`CGEvent`).
- **Network exposure:** the host opens an HTTP/WebSocket port on the LAN. Anyone on the same network could connect. Mitigate with a **shared-secret token** (`Hello{token}`) shown in the tray UI and encoded in the QR/URL; reject sessions without it.
- **Transport encryption:** WebRTC media is **always encrypted** (DTLS-SRTP). The *signaling* WebSocket is plain `ws://` on the LAN by default; optionally upgrade to `wss://` with a self-signed cert if the threat model warrants.
- **No data leaves the LAN:** no STUN/TURN, no cloud, a strong privacy story vs. commercial tools.
- **Code signing / notarization:** sign with a **Developer ID** certificate and notarize, so Gatekeeper launches it without warnings. Notarization is an *automated malware scan* and does **not** inspect for private-API use, so an app calling `CGVirtualDisplay` notarizes normally. It is **App Review**, a separate process that applies only to Mac App Store submissions, that rejects private APIs. DeskPad, BetterDummy, BetterDisplay and SimpleDisplay all ship notarized outside the store on exactly this basis.

12. Dependencies (Crates) & Build

Version note: the Apple-framework binding ecosystem moves fast. Treat the versions below as "known-good families," and run `cargo add` to pin the current release at build time. Prefer `objc2`'s official framework crates where they exist.

Concern	Recommended crate(s)	Role
Obj-C runtime / Foundation	`objc2`, `objc2-foundation`, `objc2-app-kit`	Call Apple frameworks; runtime class lookup for the private `CGVirtualDisplay`.
Obj-C blocks / dispatch	`block2`, `dispatch2` (or libdispatch FFI)	terminationHandler block; dispatch queues for descriptor + SCStream.
Capture + media types	`objc2-screen-capture-kit`, `objc2-core-media`, `objc2-core-video`, or `cidre`	ScreenCaptureKit + CVPixelBuffer/CMSampleBuffer. `cidre` also wraps VideoToolbox richly.
Video encode	`cidre` (VideoToolbox) or raw `extern "C"` FFI	VTCompressionSession is a C API; `cidre` gives the smoothest Rust ergonomics.
WebRTC	`webrtc` (webrtc-rs)	Pure-Rust peer connection, tracks, DataChannels, RTP.
Async + HTTP/WS signaling	`tokio`, `axum`, `tokio-tungstenite`	Runtime, static page server, WebSocket signaling on one port.
Serialization	`serde`, `serde_json`	Signaling + config messages.
Tray UI + QR	`tray-icon`, `muda`, `qrcode`	Menu-bar app, connect URL, QR code.
Native client (phase 2)	`winit`, `wgpu`, `webrtc`, decoder via `windows` / `ffmpeg-next`	Windows full-screen renderer + hardware decode.

webrtc-rs 0.11 cannot complete a DTLS handshake with current Chrome. Discovered at M3, 27 July 2026. Modern Chrome offers post-quantum hybrid key agreement in its DTLS ClientHello, and `webrtc-dtls` 0.10 knows only P256, P384 and X25519. It fails with `Failed to start manager dtls: invalid named curve`, and the symptom is misleading: ICE reaches *connected* and signalling looks healthy, so it presents as media that never arrives rather than as a handshake error. **Use webrtc 0.17.2 or newer**, which handles unknown curves. This is a good argument for pinning deliberately rather than tracking a version family: the failure is invisible without turning on `webrtc_dtls` logging.

12.1 Toolchain & build

```
# prerequisites: Xcode command-line tools (for the macOS SDK), stable Rust
rustup toolchain install stable
# build the host (macOS)
cargo build -p host --release
# run (grants Screen Recording prompt on first launch)
cargo run  -p host
```

Linking the private API: because we resolve `CGVirtualDisplay` classes with `AnyClass::get("…")` at runtime, there is *no* link-time symbol to satisfy, since CoreGraphics is already loaded, so no special linker flags are needed. Screen Recording/Accessibility entitlements are handled via the app bundle's `Info.plist` usage strings and TCC prompts.

13. Risks & Mitigations

Risk	Impact	Mitigation
Private API breaks on a macOS update	Virtual display fails to create	Isolated in one crate; mirror DeskPad/BetterDummy (actively maintained); pin/test per macOS version; fall back to an $8 HDMI dummy-plug path (100% public APIs). Note this risk is about *stability across macOS releases* and is untouched by the decision to skip the App Store (§2.2): skipping the store removes a distribution constraint, not a breakage constraint, so the one-crate isolation rule still earns its keep.
objc2 / cidre API churn	Build breaks on upgrade	Pin exact versions in `Cargo.lock`; upgrade deliberately.
VideoToolbox FFI complexity	Slow encoder milestone	Use `cidre`'s VT wrappers instead of hand-rolled FFI; validate with a local decode loop first.
Wi-Fi latency/jitter	Laggy feel	5 GHz/Wi-Fi 6; drop-oldest back-pressure; offer wired/USB-tether path; adaptive bitrate.
Browser codec quirks	Black video on some clients	Default to H.264 Constrained Baseline, Annex-B, in-band SPS/PPS; HEVC only when negotiated.
Run-loop / thread-affinity bugs	Crashes, no frames	Strict main-thread rule for Apple APIs; channels to tokio; never call AppKit off-main.
Ghost displays after crash	Leftover virtual monitors	RAII `Drop` + a termination handler; on launch, reclaim/cleanup stale displays.
Input coordinate mapping (phase 2)	Cursor offset/misclick	Map through the display's global origin + HiDPI scale; test across arrangements.

14. Roadmap & Milestones

Sequenced to **de-risk early** and produce a visible demo at each step. Estimates assume one developer comfortable with Rust, learning the macOS specifics.

#	Milestone	Deliverable / demo	Rough effort
M0	Virtual-display spike **(done)**	Verified 27 July 2026 on macOS 26.5: display created, visible to public CoreGraphics, removed cleanly with no ghost. See `crates/virtual-display`.	done
M1	Capture to disk **(done)**	Verified 27 July 2026: ScreenCaptureKit delivers BGRA frames from the virtual display, written to PNG. See `crates/capture`.	done
M2	Encode and verify **(done)**	Verified 27 July 2026: VideoToolbox H.264 Main, Annex-B, zero-copy from capture. ffprobe reports `has_b_frames=0` and ffmpeg decodes with zero errors. See `crates/encoder`.	done
M3	WebRTC to browser **(done)**	Verified 27 July 2026 with a real headless Chrome doing the actual handshake: connected, codec video/H264, frames decoded at 1440x900. See `crates/transport`.	done
M4	Extended display E2E **(done)**	Verified 27 July 2026: `annex` creates a virtual display, streams it, drag a window across. 57 fps, no freezes. **v1 works.**	done
M5	Interactive input	DataChannel input → `CGEvent` injection; mouse/keyboard work on the second screen.	~1 week
M6	Polish **(partly done)**	Menu bar UI and QR code done. Auth token, resolution picker, multi-client tuning and HEVC remain.	ongoing
M7	Native Rust client	`winit` + `wgpu` Windows app with HW decode for lowest latency.	1–2 weeks

M2 result, 27 July 2026: 45 frames encoded, 45 out, 0 malformed. Every keyframe carries SPS and PPS in band, so a client joining mid-stream can start at any keyframe. ffprobe independently confirms **profile Main, 1920x1080, yuv420p,** **`has_b_frames=0`**, and ffmpeg decodes the file with zero errors. That `has_b_frames=0` is external proof that `AllowFrameReordering=false` took effect, which is the single most important latency setting in the encoder. Measured around 4.4 Mbit/s on a near-static desktop against an 8000 kbps ceiling, comfortably inside the budget in section 10.

> **Fastest path to "it works":** M0 → M1 → M2 → M3 → M4. Note that M3 uses the *real* main display so you can prove the streaming half **before** depending on the private virtual-display API, so the two halves are validated independently, then joined at M4.

15. Testing & Verification Strategy

- **Unit tests** for pure logic: config parsing, signaling message (de)serialization, coordinate mapping math, back-pressure/queue behavior.
- **Encoder round-trip test:** feed a synthetic `CVPixelBuffer` (known pattern) → encode → decode → assert similarity (PSNR threshold). Catches format/Annex-B/SPS-PPS regressions.
- **Capture smoke test:** create a virtual display, render a known color, capture one frame, assert pixels.
- **Integration (headless):** host + a headless WebRTC client (webrtc-rs on the test side) negotiate and receive N frames, which verifies the full signaling + media path in CI without a browser.
- **Manual latency measurement:** display a millisecond timer on the Mac's virtual screen, photograph both screens together, subtract: the classic glass-to-glass method.
- **CI:** build + unit/integration tests on a macOS runner; the private-API creation test is gated to run only on self-hosted/interactive machines where TCC prompts can be pre-authorized.
- **Compatibility matrix:** test the browser client on Chrome, Edge, and Firefox on Windows; record the working codec/profile per browser.

Appendix A: Reverse-engineered `CGVirtualDisplay` interface

Verified 27 July 2026 on macOS 26.5. The header below was cross-checked against **DeskPad** (`CGVirtualDisplayPrivate.h`) and **BetterDummy** (`Bridging-Header.h` , branch `opensource`), and exercised by the M0 spike, which created and removed a display successfully. Earlier revisions of this appendix were written from memory and were **wrong in three places**, noted inline. Still copy from the references rather than from here when a macOS release lands.

The two references disagree, and it matters. DeskPad declares `initWithWidth:height:refreshRate:` as taking `NSUInteger` (64-bit) and `CGFloat`. BetterDummy declares `unsigned int` (32-bit) and `double`, and backs it with a class dump showing the real ivars: `unsigned int _width; unsigned int _height; double _refreshRate;`. The ivar layout settles it: **32-bit**. DeskPad's version happens to work on arm64 because the low half of the register carries any sane resolution, but it is incorrect. **Follow BetterDummy.**

```
// Private classes living inside CoreGraphics.framework (not in the SDK).
@interface CGVirtualDisplayMode : NSObject
@property(readonly, nonatomic) double refreshRate;
@property(readonly, nonatomic) unsigned int height;
@property(readonly, nonatomic) unsigned int width;
- (id)initWithWidth:(unsigned int)w height:(unsigned int)h refreshRate:(double)r;
@end                                    // ^^^^ CORRECTION 1: 32-bit, not NSUInteger

@interface CGVirtualDisplaySettings : NSObject
@property(nonatomic) unsigned int hiDPI;               // 1 = Retina backing store
@property(retain, nonatomic) NSArray *modes;
- (id)init;
@end            // CORRECTION 2: there is NO `rotation` property. Only these two.

@interface CGVirtualDisplayDescriptor : NSObject
@property(retain, nonatomic) id queue;                 // dispatch_queue_t
@property(retain, nonatomic) NSString *name;
@property(nonatomic) CGPoint whitePoint, redPrimary, greenPrimary, bluePrimary;
@property(nonatomic) unsigned int maxPixelsHigh, maxPixelsWide;
@property(nonatomic) CGSize sizeInMillimeters;         // physical size, drives DPI
@property(nonatomic) unsigned int serialNum, productID, vendorID;
@property(copy, nonatomic) id terminationHandler;      // Obj-C block
- (id)init;
- (id)dispatchQueue;                    // newer alias for the `queue` property
- (void)setDispatchQueue:(id)arg1;
@end   // CORRECTION 3: the four colour primaries above were missing entirely.

@interface CGVirtualDisplay : NSObject
@property(readonly, nonatomic) unsigned int displayID;  // feed to ScreenCaptureKit
@property(readonly, nonatomic) unsigned int hiDPI;
@property(readonly, nonatomic) NSArray *modes;
- (id)initWithDescriptor:(id)arg1;
- (BOOL)applySettings:(id)arg1;
@end
```

The non-obvious part: creating the `CGVirtualDisplay` object does **not** make a monitor appear. After `initWithDescriptor:` the object exists but has no modes, and macOS shows nothing. It is `applySettings:`, carrying at least one `CGVirtualDisplayMode`, that attaches the display. Worth knowing before spending an hour wondering why System Settings is unchanged.

Creation recipe, as implemented in `crates/virtual-display`:

1. Resolve all four classes with `AnyClass::get`. No linker flags are needed: CoreGraphics is already loaded, so there is no link-time symbol to satisfy.
2. Build the descriptor. Set the dispatch queue (a global user-interactive queue works), the name, the pixel ceiling, the physical size, the four colour primaries, and the vendor/product/serial ids.
3. `alloc` then `initWithDescriptor:`, then release the descriptor.
4. Build one `CGVirtualDisplayMode` per resolution you want to offer, and put them in an `NSArray`.
5. Build the settings object, set `hiDPI` and `modes`, then call `applySettings:`. **This is the call that makes the monitor appear.**
6. Read `displayID` off the display object and hand it to the capture crate.

Retain the display object for the whole session. Dropping it removes the monitor, which is why the Rust wrapper ties removal to `Drop`: a leaked handle leaves the user with a ghost display they cannot clear without logging out.

Appendix B: References

Reference implementations to read / adapt

- **DeskPad**: Swift virtual monitor, MIT. Gold-standard for the `CGVirtualDisplay` creation code. github.com/Stengo/DeskPad
- **BetterDummy**: Swift virtual display, MIT. Second independent reference for the private API. github.com/waydabber/BetterDummy
- **BetterDisplay**: the productized, feature-rich descendant. github.com/waydabber/BetterDisplay
- **SimpleDisplay**: a current (macOS 14+) menu-bar app doing the same `CGVirtualDisplay` trick. Useful as a third, recently-maintained cross-check on the header. **GPL-3.0**, see the licence note below. simpledisplay.app
- **Deskreen**: Electron/WebRTC browser-as-second-screen; the streaming/architecture model. github.com/pavlobu/deskreen

Licence hygiene on the virtual-display crate: DeskPad and BetterDummy are **MIT**; SimpleDisplay is **GPL-3.0**. GPL-3.0 is copyleft, so lifting its code into `crates/virtual-display` would oblige Annex to ship under GPL-3.0 as well. Read SimpleDisplay to *understand* the API and to confirm what still works on current macOS, but transcribe the header and the creation recipe from the MIT projects. None of these tools solve Half 2: SimpleDisplay's own docs tell you to reach the virtual display over VNC, Screen Sharing, or Parsec, which is precisely the gap Annex closes.

Rust libraries

- **objc2** family: Apple framework bindings + runtime. github.com/madsmtm/objc2
- **cidre**: comprehensive Rust bindings for ScreenCaptureKit, CoreMedia, VideoToolbox. github.com/yury/cidre
- **webrtc-rs**: pure-Rust WebRTC. github.com/webrtc-rs/webrtc
- **tokio / axum**, **winit / wgpu**, **tray-icon / muda**.

Apple documentation

- ScreenCaptureKit: developer.apple.com/documentation/screencapturekit
- VideoToolbox (VTCompressionSession): developer.apple.com/documentation/videotoolbox
- Quartz Display Services / CGEvent: developer.apple.com/documentation/coregraphics

Appendix C: Glossary

Term	Meaning
CGVirtualDisplay	Private CoreGraphics class that creates a software "monitor" macOS treats as real.
CGDirectDisplayID	Numeric identifier macOS assigns each display; the handle passed between capture and injection.
ScreenCaptureKit (SCK)	Apple's modern, GPU-accelerated screen-capture framework (macOS 12.3+).
CVPixelBuffer	A GPU-friendly image buffer; the frame format passed capture → encoder.
VideoToolbox (VT)	Apple's hardware video encode/decode framework.
NAL unit / Annex-B	H.264 packaging; Annex-B is the start-code stream WebRTC's payloader expects.
WebRTC	Real-time media/data transport with built-in encryption, NACK, and congestion control.
SDP / ICE	WebRTC's session description and connectivity-establishment protocols (exchanged via signaling).
DataChannel	WebRTC's arbitrary-data channel; carries phase-2 input events.
Signaling	The out-of-band exchange (our WebSocket) that bootstraps a WebRTC connection.
TCC	macOS "Transparency, Consent & Control": the permission system behind Screen Recording/Accessibility prompts.
RAII / Drop	Rust pattern where releasing a value frees its resource; here, removing the virtual display.
Glass-to-glass latency	Time from a change on the Mac to it appearing on the Windows screen.
HiDPI	Retina-style rendering where the backing store has 2× the logical pixels.